

## سوغاتی

«آمارو» در حال خرید سوغاتی در یک مغازه است. در کل  $N$  نوع مختلف سوغاتی وجود دارد. در این مغازه از هر نوع سوغاتی به تعداد بی‌نهایت موجود است.

هر نوع سوغاتی قیمت مشخصی دارد. یعنی، هر سوغاتی از نوع  $i$  ( $0 \leq i < N$ ) قیمتی معادل  $P[i]$  سکه دارد، که در آن  $P[i]$  یک عدد صحیح مثبت است.

آمارو می‌داند که قیمت سوغاتی‌ها به ترتیب نزولی مرتب‌اند و همچنین متمایز هستند. به طور مشخص،  $P[0] > P[1] > \dots > P[N-1] > 0$ . علاوه بر این، او توانسته مقدار  $P[0]$  را پیدا کند. متأسفانه، آمارو هیچ اطلاعات دیگری در مورد قیمت سوغاتی‌ها ندارد.

برای خرید سوغاتی، آمارو تعدادی معامله با فروشنده انجام می‌دهد.

هر معامله شامل مراحل زیر است:

1. آمارو تعدادی (مثبت) سکه به فروشنده می‌دهد.
  2. فروشنده این سکه‌ها را در اتاق پشتی روی یک میز روی هم می‌چیند، طوری که آمارو نمی‌تواند آن‌ها را ببیند.
  3. فروشنده هر نوع سوغاتی را یکی یکی، به ترتیب  $0, 1, \dots, N-1$  در نظر می‌گیرد (از کوچک به بزرگ). هر نوع سوغاتی دقیقاً یک بار در هر معامله در نظر گرفته می‌شود.
    - هنگام بررسی نوع سوغاتی  $i$ ، اگر تعداد فعلی سکه‌های موجود حداقل  $P[i]$  باشد، آنگاه
      - فروشنده  $P[i]$  سکه از سکه‌های روی میز برمی‌دارد، سپس
      - فروشنده یک سوغاتی از نوع  $i$  روی میز می‌گذارد.
  4. فروشنده تمام سکه‌های باقی‌مانده و تمام سوغاتی‌های روی میز را به آمارو می‌دهد.
- توجه داشته باشید که قبل از شروع معامله، هیچ سکه یا سوغاتی روی میز نیست.
- وظیفه شما این است که آمارو را راهنمایی کنید تعدادی معامله انجام دهد، به طوری که:

- در هر معامله او حداقل یک سوغاتی بخرد، و
- در مجموع همه معامله‌ها، دقیقاً  $i$  عدد از سوغاتی نوع  $i$  (به ازای هر  $i$  که  $0 \leq i < N$ ) خریداری کند. توجه داشته باشید که این بدان معناست که آمارو نباید هیچ سوغاتی از نوع 0 بخرد.

نیازی نیست که آمارو تعداد معامله‌ها را کمینه کند.

## جزئیات پیاده‌سازی

شما باید تابع زیر را پیاده‌سازی کنید.

```
void buy_souvenirs(int N, long long P0)
```

- $N$ : تعداد انواع سوغاتی.
- $P_0$ : مقدار  $P[0]$ .
- این تابع دقیقاً یک بار به ازای هر تست کیس اجرا می شود.

تابع بالا می تواند تابع زیر را برای دستور دادن به آمارو جهت انجام یک معامله فراخوانی کند:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- $M$ : تعداد سکه هایی که آمارو به فروشنده تحویل می دهد.
- این تابع یک `pair` را برمی گرداند. اولین عنصر این `pair`، آرایه ای به نام  $L$  است، که شامل انواع سوغاتی هایی است که در این معامله خریداری شده اند (به ترتیب صعودی). عنصر دوم یک عدد صحیح  $R$  است، تعداد سکه هایی که پس از معامله به آمارو بازگردانده شده است.
- لازم است که  $P[0] > M \geq P[N-1]$ . شرط  $P[0] > M$  تضمین می کند که آمارو هیچ سوغاتی از نوع 0 نمی خرد، و  $M \geq P[N-1]$  تضمین می کند که آمارو حداقل یک سوغاتی می خرد. اگر این شرایط برقرار نباشند، راه حل شما با خطای `Output isn't correct: Invalid argument` مواجه خواهد شد. توجه داشته باشید که برخلاف  $P[0]$ ، مقدار  $P[N-1]$  در ورودی ارائه نشده است.
- این تابع را می توان حداکثر 5000 بار در هر تست کیس فراخوانی کرد.

رفتار گریدر آدپتیو نیست. این یعنی توالی قیمت های  $P$  قبل از فراخوانی `buy_souvenirs` ثابت است.

## محدودیت ها

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$  برای هر  $i$  که  $0 \leq i < N$ .
- برای هر  $i$  که  $0 \leq i < N-1$  باشد  $P[i] > P[i+1]$ .

## زیرمسئله ها

| Subtask | Score | Additional Constraints                                                                                                                        |
|---------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | 4     | $N = 2$                                                                                                                                       |
| 2       | 3     | $P[i] = N - i$ for each $i$ such that $0 \leq i < N$ .                                                                                        |
| 3       | 14    | $P[i] \leq P[i+1] + 2$ for each $i$ such that $0 \leq i < N-1$ .                                                                              |
| 4       | 18    | $N = 3$                                                                                                                                       |
| 5       | 28    | $P[i+1] + P[i+2] \leq P[i]$ for each $i$ such that $0 \leq i < N-2$ .<br>$P[i] \leq 2 \cdot P[i+1]$ for each $i$ such that $0 \leq i < N-1$ . |
| 6       | 33    | No additional constraints.                                                                                                                    |

## نمونه

فراخوانی زیر را در نظر بگیرید.

```
buy_souvenirs(3, 4)
```

تعداد انواع سوغاتی  $N = 3$  است و  $P[0] = 4$  است. توجه داشته باشید که فقط سه توالی ممکن از قیمت‌های  $P$  وجود دارد:  $[4, 3, 2]$ ،  $[4, 3, 1]$ ، و  $[4, 2, 1]$ .

فرض کنید که `buy_souvenirs` تابع `transaction(2)` را فراخوانی می‌کند. فرض کنید فراخوانی مقدار  $([2], 1)$  را برمی‌گرداند، یعنی آمارو یک سوغاتی از نوع 2 خریده است و فروشنده 1 سکه به او پس داده است. توجه داشته باشید که این به ما امکان می‌دهد نتیجه بگیریم که  $P = [4, 3, 1]$ ، زیرا:

- برای  $P = [4, 3, 2]$ ، `transaction(2)` مقدار  $([2], 0)$  را برمی‌گرداند.
- برای  $P = [4, 2, 1]$ ، `transaction(2)` مقدار  $([1], 0)$  را برمی‌گرداند.

سپس `buy_souvenirs` می‌تواند `transaction(3)` را فراخوانی کند، که مقدار  $([1], 0)$  را برمی‌گرداند، یعنی آمارو یک سوغاتی از نوع 1 خریده است و فروشنده 0 سکه به او پس داده است. تا اینجا، در مجموع، او یک سوغاتی از نوع 1 و یک سوغاتی از نوع 2 خریده است.

در نهایت، `buy_souvenirs` می‌تواند `transaction(1)` را فراخوانی کند، که مقدار  $([2], 0)$  را برمی‌گرداند، یعنی آمارو یک سوغاتی از نوع 2 خریده است. توجه داشته باشید که می‌توانستیم از `transaction(2)` نیز در اینجا استفاده کنیم. در این مرحله، در مجموع آمارو، یک سوغاتی از نوع 1 و دو سوغاتی از نوع 2 دارد، که مطابق هدف است.

## گرایدر نمونه

قالب ورودی:

```
N
P[0] P[1] ... P[N-1]
```

قالب خروجی:

```
Q[0] Q[1] ... Q[N-1]
```

در اینجا  $Q[i]$  تعداد کل سوغاتی‌های خریداری‌شده از نوع  $i$  (به ازای هر  $0 \leq i < N$ ) است.